Software Requirements Specification

for

<ClauseGuard: AI-Powered Contract Risk Analyzer for Indian MSMEs>

Version 1.0 approved

Prepared by <Tanmaya Gupta>

<VIT Chennai>

<15/07/2026>

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction 1

1.1 Purpose 1

1.2 Document Conventions 1

1.3 Intended Audience and Reading Suggestions 1

1.4 Product Scope 1

1.5 References 2

2. Overall Description 2

2.1 Product Perspective & System Architecture 2

2.2 Product Functions 4

2.3 User Classes and Characteristics 4

2.4 Operating Environment 4

2.5 Design and Implementation Constraints 4

2.6 User Documentation 5

2.7 Assumptions and Dependencies 5

3. External Interface Requirements 5

3.1 User Interfaces 5

3.2 Hardware Interfaces 6

3.3 Software Interfaces 6

3.4 Communications Interfaces 6

4. System Features 6

4.1 Contract Upload and Parsing 6

4.2 Clause Segmentation 7

4.3 Clause-Type Classification 7

4.4 Risk Scoring 7

4.5 Report Generation and Export 8

4.6 User Account and Contract History 8

5. Other Nonfunctional Requirements 8

5.1 Performance Requirements 8

5.2 Safety Requirements 9

5.3 Security Requirements 9

5.4 Software Quality Attributes 9

5.5 Business Rules 9

6. Other Requirements 10

Appendix A: Glossary 10

Appendix B: Analysis Models 10

Appendix C: To Be Determined List 11

1. Introduction

1.1 Purpose

This document specifies the software requirements for ClauseGuard, an AI-powered contract clause classification and risk-flagging tool aimed at Indian Micro, Small, and Medium Enterprises (MSMEs). This SRS covers Version 1.0 of the product, which is the complete system: contract upload and parsing, clause segmentation, clause-type classification, risk scoring, and plain-language report generation. This document describes the full scope of the system as a standalone application; it does not describe any larger enterprise system into which it may later be integrated.

1.2 Document Conventions

Functional requirements are labeled sequentially as REQ-X.Y within each System Feature (e.g., REQ-1.1, REQ-1.2), where X is the feature number. Priority for each System Feature is stated as High, Medium, or Low. Individual requirements within a feature inherit the priority of the parent feature unless explicitly stated otherwise.

The terms "the system," "the tool," and "ClauseGuard" are used interchangeably to refer to the product being specified. The term "SME user" refers to the small business owner or authorized staff member using the tool; "clause" refers to a discrete contractual provision within an uploaded document. TBD marks information not yet finalized; all TBD items are tracked in Appendix C.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- **Developers/engineers** implementing the system — should read Sections 3, 4, and 5 in detail.
- **Project evaluators/course instructors** assessing scope and design justification — should read Sections 1, 2, and 4.
- **ML/data team members** — should focus on Sections 2.5, 4.1–4.3, and Appendix B.
- **Testers** — should read Section 4 (functional requirements and stimulus/response sequences) closely to derive test cases.
- **Prospective end users (SME owners)** consulted during requirements validation — Sections 1.4, 2.2, and 2.3 give a plain overview.

Readers new to the project should start with Section 1 (Introduction) and Section 2 (Overall Description) before proceeding to the detailed requirements in Sections 3–5.

1.4 Product Scope

Indian MSMEs routinely sign vendor, supply, NDA, and service contracts without access to in-house legal counsel. Risky clauses — one-sided indemnity, silent auto-renewal, unilateral termination rights, unlimited liability, weak payment-timeline protection — often go unnoticed until a dispute arises, by which point the cost of resolution is far higher than the cost of a pre-signature review would have been.

ClauseGuard addresses this by allowing an SME user to upload a contract (PDF or DOCX) and receive an automated, clause-by-clause breakdown that:

1. Segments the document into individual clauses.
2. Classifies each clause by type (e.g., indemnity, termination, payment terms, liability, confidentiality).
3. Scores each clause's risk level (safe / caution / high-risk) relative to a defensible fairness rubric.
4. Generates a plain-language report explaining why a flagged clause is risky, so a non-lawyer can decide whether it's worth paying for professional legal review.

The product's goal is not to replace a lawyer, but to triage: cheaply and quickly identify which 2–3 clauses in a 20-clause contract actually deserve a paid legal opinion, and give the SME owner enough context to negotiate or ask informed questions in the meantime.

1.5 References

1. Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. https://arxiv.org/abs/2103.06268
2. Lippi, M. et al. (2019). CLAUDETTE: an Automated Detector of Potentially Unfair Clauses in Online Terms of Service. Artificial Intelligence and Law. https://arxiv.org/abs/1805.01217
3. Singh, Joshi, Jiang & Paik (2025). A Survey of Classification Tasks and Approaches for Legal Contracts. Artificial Intelligence Review. https://arxiv.org/abs/2507.21108
4. Is It Worth Using LLMs for Unfair Clause Detection in Terms of Service? AI & Law 2025 Conference Proceedings. https://dl.acm.org/doi/full/10.1145/3769126.3769218
5. Chalkidis, I., Fergadiotis, M., Malakasiotis, P., & Androutsopoulos, I. (2020). LEGAL-BERT: The Muppets Straight Out of Law School. EMNLP Findings. https://aclanthology.org/2020.findings-emnlp.261/
6. The Atticus Project — CUAD dataset documentation, Hugging Face. https://huggingface.co/datasets/theatticusproject/cuad
7. Indian Contract Act, 1872 (reference for domain norms governing Indian commercial contract clauses).

2. Overall Description

2.1 Product Perspective & System Architecture

ClauseGuard is a new, self-contained, full-stack web application. It is not a follow-on to an existing internal system. It integrates an original machine learning pipeline built on top of a public benchmark dataset (CUAD) and an original risk-classification layer inspired by, but distinct from, the CLAUDETTE/UNFAIR-ToS body of work, which addressed consumer Terms-of-Service rather than B2B vendor/supply contracts in the Indian commercial context.

The systemic operational flow and component interactions are illustrated below:

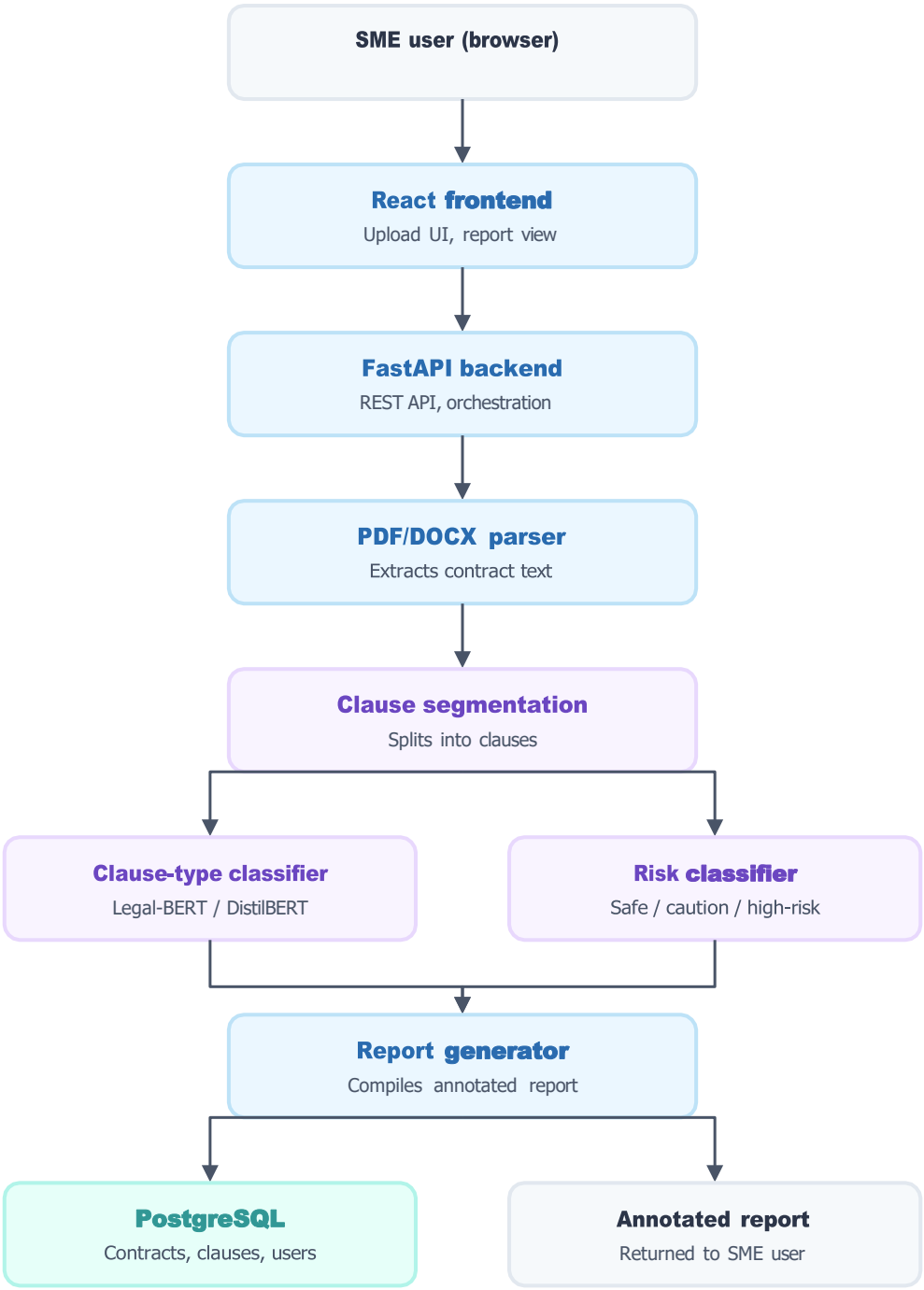


Figure 1: ClauseGuard System Functional Pipeline & Architectural Layout

2.2 Product Functions

At a high level, ClauseGuard allows a user to perform the following core activities:

- Register/log in and manage an account securely.
- Upload a contract file in PDF or DOCX format.
- Trigger automated clause segmentation of the uploaded document.
- View each clause classified accurately by legal type (e.g., indemnity, termination, payment terms).

- View an objective risk label (safe / caution / high-risk) assigned per clause, accompanied by a clear, plain-language contextual explanation.
- View a summarized, color-coded, clause-by-clause comprehensive report of the entire contract.
- Download the resulting annotated report seamlessly as a PDF document.
- Track historical operational reviews through a dedicated dashboard displaying "contracts reviewed over time."

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise	Frequency of Use
SME Owner / Primary User	Small business owner or authorized staff member signing vendor/supply contracts. No formal legal background assumed.	Low–Medium (general web application usage literacy, not technical or legal).	Occasional to moderate (correlated with new operational contract execution cycles).
Admin / Operator	Maintains platform services, monitors real-world model prediction shifts, manages enterprise user accounts.	High (technical engineering and operations background).	Frequent (ongoing daily administration).
Reviewer / Evaluator	Course project instructor or panel evaluating systemic architectural correctness and code design parameters.	High (expert/academic baseline).	One-time or periodic evaluation rounds.

The SME Owner represents the primary user class; the system's human-computer interface, plain language engine, and output analytics formatting must be constructed entirely for non-lawyers.

2.4 Operating Environment

- **Client Side:** Compatible with all modern responsive web browsers (Chrome, Firefox, Edge, Safari — latest two major releases) across desktop, tablet, and mobile breakpoints.
- **Server Side:** Linux-based cloud infrastructure runtime stack hosting an environment built with Python 3.10+.
- **Model Inference:** Optimized CPU-based model inference execution routines for academic demo scaling; GPU parallel processing configurations optional during massive model fine-tuning developmental workflows.
- **Database Instance:** Persistent relational engine running PostgreSQL 14+.

2.5 Design and Implementation Constraints

- The primary clause-type multi-label classifier will be built by fine-tuning pre-trained Legal-BERT or DistilBERT architectures, constraining the core ML layer to a transformer-focused PyTorch/Hugging Face framework ecosystem.
- The primary underlying public baseline dataset (CUAD) remains intrinsically US-centric, long-form, and corporate M&A focused. This baseline constraints native accuracy parameters against localized, short-form Indian commercial vendor/supply documents, creating an implementation requirement for a localized domain-adaptation corpus consisting of Indian contract templates.
- The risk evaluation output engine must maintain complete explainability and compliance auditable records; a black-box percentage score remains insufficient. Every operational risk assignment flag must be dynamically mapped directly to a transparent rubric guideline.
- The application does not perform formal legal representation or output formal legal advice, meaning the system boundary copy must strictly represent itself as an educational triage tool.
- Backend engineering must utilize Python (FastAPI); frontend structural frameworks are restricted to React.
- Academic project timelines limit the initial pipeline taxonomy scope to approximately 15–20 core clause classes and a 3-tier risk grading matrix.

2.6 User Documentation

The system will deliver the following user support assets alongside production binaries:

- An integrated, step-by-step interactive first-use tooltip UI guide.
- A digital User Guide outlining clean contract scanning techniques and risk threshold interpretations.
- An embedded natural plain-language glossary corresponding strictly with legal concepts detailed in Appendix A.
- An explicit FAQ system clarifying operational boundaries (e.g., explicit statements detailing the non-lawyer tool parameters).

2.7 Assumptions and Dependencies

- Assumes uploaded contract assets are strictly delivered in readable English text-extractable PDF/DOCX layouts without scanned raster image matrices.
- Depends entirely on the ongoing open availability and licensing criteria of the Atticus Project CUAD baseline dataset repository.
- Depends on compiling an auxiliary domain adaptation corpus composed of real-world or standard template Indian commercial agreement forms.
- Assumes third-party utility parsing frameworks (e.g., PyMuPDF, python-docx) maintain backward runtime compatibility.

3. External Interface Requirements

3.1 User Interfaces

- **Upload Screen:** Minimalist visual element supporting secure drag-and-drop or local file system selection, containing active size-validation error warnings.
- **Processing Screen:** Deterministic visual loader detailing live extraction, segmentation, and machine learning computation states.
- **Report Screen:** Sequential linear list layout displaying isolated extracted clause blocks, explicitly color-coded via a categorical indicator spectrum (Green = Safe, Yellow = Caution, Red = High-Risk), paired with interactive panels showing plain explanations.
- **Dashboard View:** Administrative entry panel providing quick access to account history, metrics tracking files analyzed, and summary category distributions.
- **Export Control:** Global action triggering automated generation and formatting of clear print-ready downloadable PDFs.

3.2 Hardware Interfaces

Not applicable. The system does not possess direct hardware integrations or peripheral equipment dependencies outside standard internet-connected client systems and standard server hardware nodes.

3.3 Software Interfaces

Component Name	Version Spec	Purpose / Contextual Integration
FastAPI	Python 3.10+	Asynchronous backend REST API framework handling client routing, database transactions, and model coordination.
React	Latest Stable	Client-side single-page UI layer rendering responsive views and state controls.
PostgreSQL	14+	Relational database storage engine capturing structured user records, file tracks, clause slices, and report states.
PyMuPDF / pdfplumber	Latest Stable	Text utility frameworks extracting text layers from raw digital PDF streams.
python-docx	Latest Stable	Document processing module parsing DOCX XML elements.
Hugging Face Transformers	PyTorch Core	Deep learning lifecycle orchestration handling transformer text tokenizer mappings and raw inference logic execution.
Legal-BERT / DistilBERT	Fine-tuned	Pre-trained language backbone weights adapted specifically for classification tasks.

3.4 Communications Interfaces

All modern communications endpoints strictly require web-socket encryption over HTTPS via TLS 1.3 standards. Cross-origin data transfer structures utilize explicit JSON formats via defined REST application boundaries. Client authorization mappings utilize standard stateless JSON Web Tokens (JWT) signed and validated via server cryptography keys.

4. System Features

4.1 Contract Upload and Parsing

4.1.1 Description and Priority

Provides mechanisms enabling end-users to input local target document payloads into the application pipeline to undergo deep formatting extraction. Priority: High.

4.1.2 Stimulus/Response Sequences

- **Stimulus:** User drops an extractable PDF contract file onto the target web drop area.
- **Response:** The client triggers client-side size checks, transfers the payload via POST request, and the backend extracts the text stream, outputting a verification code while initiating the segmentation process.
- **Stimulus:** User uploads an absolute image raster/scanned non-text PDF document block.
- **Response:** The text extraction logic detects empty character maps and outputs an unambiguous non-technical warning detailing the error.

4.1.3 Functional Requirements

> **REQ-1.1:** The system shall accept contract uploads in PDF and DOCX format only.

> **REQ-1.2:** The system shall reject files exceeding [TBD-1] MB with a clear, non-technical error message.

> **REQ-1.3:** The system shall extract raw text from the uploaded file while preserving paragraph/clause boundaries where structurally possible.

> **REQ-1.4:** If text extraction yields no usable content (e.g., a scanned/image-only PDF), the system shall notify the user that the file could not be processed and suggest uploading a text-based version.

> **REQ-1.5:** The system shall store the original uploaded file and extracted text against the user's account for later retrieval.

4.2 Clause Segmentation

4.2.1 Description and Priority

Divides large raw textual document strings into isolated, contextually distinct paragraph clause candidates. Priority: High.

4.2.2 Stimulus/Response Sequences

- **Stimulus:** Extraction service emits raw document text strings.
- **Response:** System executes deterministic layout rules and regex numbering sweeps to transform text blocks into an ordered array of clause candidates.

4.2.3 Functional Requirements

> **REQ-2.1:** The system shall segment contract text into individual clauses using a combination of rule-based heuristics (e.g., numbering patterns, headings) and a sentence-boundary detection model.

> **REQ-2.2:** Each segmented clause shall be assigned a unique identifier and retain its original position/order in the document.

> **REQ-2.3:** The system shall handle documents with non-standard formatting (e.g., missing numbering) by falling back to paragraph-level segmentation.

> **REQ-2.4:** The system shall flag clauses that are ambiguous to segment (e.g., unusually long or malformed) for manual review rather than silently misclassifying them.

4.3 Clause-Type Classification

4.3.1 Description and Priority

Applies fine-tuned neural network classification to evaluate the legal function of each candidate text string. Priority: High.

4.3.2 Stimulus/Response Sequences

- **Stimulus:** Core backend feeds isolated clause candidate text into the neural network wrapper.
- **Response:** Model serves categorical label predictions alongside normalized confidence value arrays.

4.3.3 Functional Requirements

> **REQ-3.1:** The system shall classify each clause into one or more of approximately 15–20 clause-type categories derived from the CUAD taxonomy.

> **REQ-3.2:** The system shall support multi-label classification, since a single clause may belong to more than one category.

> **REQ-3.3:** The system shall report a confidence score for each predicted clause type.

> **REQ-3.4:** If no clause type can be predicted above a minimum confidence threshold [TBD-2], the clause shall be labeled "Uncategorized" and surfaced to the user as such rather than guessed.

4.4 Risk Scoring

4.4.1 Description and Priority

Evaluates clauses against standard fairness criteria and applies risk tags based on unbalanced text indicators. Priority: High.

4.4.2 Stimulus/Response Sequences

- **Stimulus:** Categorized clause array enters the programmatic rubric evaluation logic loop.
- **Response:** System returns an explicit safe/caution/high-risk evaluation tag along with targeted structural rationales.

4.4.3 Functional Requirements

> **REQ-4.1:** The system shall assign each clause one of three risk levels: Safe, Caution, or High-Risk.

> **REQ-4.2:** The risk-scoring rubric shall be based on identifiable, documented criteria (e.g., unilateral termination rights, uncapped liability, absence of a cure period, auto-renewal without notice) rather than an unexplained model score.

> **REQ-4.3:** For every clause labeled Caution or High-Risk, the system shall generate a plain-language explanation of why the clause was flagged, avoiding legal jargon.

> **REQ-4.4:** The system shall allow the risk rubric/criteria to be updated or extended without requiring a full model retrain (i.e., rule-layer criteria should be configurable).

4.5 Report Generation and Export

4.5.1 Description and Priority

Compiles internal multi-tier relational schema analytics into clean visual review representations. Priority: Medium.

4.5.2 Stimulus/Response Sequences

- **Stimulus:** End-user clicks the document "Download PDF Report" selector.
- **Response:** System instantiates print-ready formatting engines to compile the text into a clean PDF document.

4.5.3 Functional Requirements

> **REQ-5.1:** The system shall generate an in-app, clause-by-clause report showing clause text, predicted type, risk label, and explanation.

> **REQ-5.2:** The system shall color-code clauses by risk level in the report view (e.g., green/yellow/red).

> **REQ-5.3:** The system shall allow the user to export the full report as a downloadable PDF.

> **REQ-5.4:** The system shall display an overall contract summary (e.g., count of high-risk clauses) at the top of the report.

4.6 User Account and Contract History

4.6.1 Description and Priority

Manages system account control, multi-tenant workspace partitioning, and storage of analyzed file structures. Priority: Medium.

4.6.2 Stimulus/Response Sequences

- **Stimulus:** Logged-in client enters the main overview portal page.
- **Response:** App pulls relational tracking history tables and builds the dashboard view listing past documents.

4.6.3 Functional Requirements

> **REQ-6.1:** The system shall allow users to create an account and log in securely.

> **REQ-6.2:** The system shall associate each uploaded contract and its report with the authenticated user's account.

> **REQ-6.3:** The system shall display a history/dashboard view listing past contracts with upload date and high-level risk summary.

> **REQ-6.4:** The system shall allow a user to re-open and re-download a past report without re-uploading the file.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall complete clause segmentation and classification for a typical contract (up to ~20 pages / ~50 clauses) within [TBD-3] seconds under normal server load.
- The system shall support concurrent processing of at least [TBD-4] simultaneous contract uploads without significant performance degradation.

5.2 Safety Requirements

Not directly applicable in the physical-harm sense. However, the system carries an informational-safety concern: incorrect or misleading risk classification could lead an SME user to sign a genuinely one-sided contract believing it is safe. To mitigate this:

- The system shall never assert 100% certainty in a risk label; confidence indicators shall be shown where relevant.
- The system shall include a persistent disclaimer that it does not constitute legal advice and that high-stakes contracts should be reviewed by a licensed professional.

5.3 Security Requirements

- The system shall encrypt all data in transit via HTTPS/TLS.
- The system shall store uploaded contracts and user data securely, with access restricted to the authenticated owner of the data.
- The system shall implement authentication using industry-standard practices (e.g., hashed passwords, token-based sessions).
- The system shall not share or expose uploaded contract content to any user other than the uploading account holder.
- Given that contracts may contain commercially sensitive information, the system shall support account-level deletion of uploaded contracts and associated data on user request.

5.4 Software Quality Attributes

- **Explainability:** Every risk flag must be accompanied by a human-readable rationale; this is treated as a first-class non-functional requirement, not an optional nicety, given the target non-lawyer audience.
- **Usability:** The interface shall be usable by users with no legal training and general (not expert) digital literacy; plain language shall be used throughout, avoiding legal jargon in the UI and reports.

- **Reliability:** The classification pipeline shall degrade gracefully — if the risk classifier fails on a clause, the system shall still display the clause-type result and mark risk as "Unable to assess" rather than failing the whole report.
- **Maintainability:** The clause-type and risk-classification models shall be swappable/retrainable independently of the surrounding application logic.
- **Portability:** The backend and model-serving components shall be containerized (e.g., Docker) to support deployment across environments.

5.5 Business Rules

- The system shall not represent its output as formal legal advice; all reports shall carry a disclaimer to this effect.
- Only the account holder who uploaded a contract (or an admin, for support purposes) may view that contract's content and report.
- Risk-scoring criteria shall be documented and auditable; any change to the rubric shall be version-tracked so that historical reports remain explainable against the rubric version used at the time.

6. Other Requirements

- **Legal/Domain requirement:** Clause-type and risk-scoring categories shall be reviewed for relevance to Indian commercial contract norms (e.g., GST-related payment clauses, Indian Contract Act conventions), not only US-centric CUAD categories.
- **Data requirement:** The system's training pipeline shall document dataset provenance (CUAD, Ref. 1; supplementary Indian contract templates) and any preprocessing/label-mapping performed, for reproducibility.
- **Internationalization:** Out of scope for V1.0 — the system supports English-language contracts only.
- **Reuse objective:** The clause-type classifier and risk-scoring layer shall be built as decoupled modules so that either could, in principle, be reused or extended independently (e.g., risk-scoring layer applied to a different contract domain in future work).

Appendix A: Glossary

Term	Definition
MSME/SME	Micro, Small and Medium Enterprise — a small business (the end user of this tool), not a legal-aid provider.
Clause	A discrete contractual provision within a contract (e.g., a single indemnity or termination provision).
Clause-type classification	The task of labeling a clause according to its legal category (e.g., indemnity, termination, liability).
Risk/fairness classification	The task of labeling a clause as standard/balanced or one-sided/risky relative to a defined rubric.
CUAD	Contract Understanding Atticus Dataset — a public, expert-annotated dataset of 500+ commercial contracts across 41 clause categories, used as the base dataset for clause-type classification.
CLAUDETTE / UNFAIR-ToS	Prior research system and dataset for detecting unfair clauses in consumer Terms of Service; the closest precedent for this project's risk-scoring layer.
Legal-BERT	A BERT variant pretrained on legal text, used as the candidate backbone for the clause classifiers.
Indemnity clause	A clause allocating responsibility for compensating losses/damages between contracting parties.
Termination-for-convenience	A clause allowing one or both parties to end a contract without cause, subject to notice terms.
Auto-renewal clause	A clause that automatically extends a contract's term unless a party opts out within a specified window.

Appendix B: Analysis Models

The system functional operational architecture flows naturally through a continuous sequence model as detailed in **Figure 1 (Section 2.1)**. The entity relationships within the database schema layer map 1-to-many configurations between User accounts and Contract documents, which sequentially map to a 1-to-many relationship with segmented Clause lists. These clauses dynamically reference individual classification records and report logs generated during processing loops.

Appendix C: To Be Determined List

#TBD	Item	Location	Description
1	Maximum accepted file size for upload	REQ-1.2	The concrete file size capacity limitation threshold for standard user document ingestion.
2	Minimum confidence threshold	REQ-3.4	The predictive score floor required before routing to the "Uncategorized" label structure.
3	Target processing time	Section 5.1	The latency budget standard for processing up to 50 parsed paragraph nodes.
4	Target number of concurrent uploads	Section 5.1	The baseline horizontal scaling metrics required for multi-tenant backend provisions.
5	Exact final list of Indian templates	Section 2.7	The specific corporate registries and legal data portal endpoints sourced for domain adaptation data sets.
6	Detailed UI wireframes / style guide	Section 3.1	Separate comprehensive project design schema specification track document reference.
7	OCR Support Inclusion	Section 2.7, REQ-1.4	Decision parameters establishing whether static visual character maps undergo extraction layers in V1.0.